M-CLTR: Measurable Controlled Latent Trajectory Reasoning

Sohan Poudel
forreserachonlyso@gmail.com
ORCID: 0009-0004-5762-3898

Transparency and Provenance Note

This manuscript is an exploratory research draft and should not be interpreted as a peer-reviewed claim of novelty, priority, or superior performance. Its purpose is to document a diagnostic experimental program on latent reasoning control.

Large language models, including ChatGPT, Gemini, and Grok, assisted with brainstorming, drafting, editing, code scaffolding, and result interpretation. The experimental direction, code execution, result selection, and final framing were determined and verified by the author.

Readers should therefore treat this paper as an early proof-of-concept report whose empirical claims depend on the included synthetic experiments and should be independently verified before they are treated as established scientific conclusions.

For additional transparency, the author also provides supporting materials, including chat conversations and code used to run the experiments.

- Summarized ChatGPT conversation: https://chatgpt.com/share/69f20851-0c10-8322-b59b-408d8ab4a9e8
- Original Gemini conversation: https://gemini.google.com/share/3cee19ece8bc
- Original Grok conversation: https://grok.com/share/bGVnYWN5LWNvcHk_4c490983-ae6f-4dd5-a1c4-2f26aa3b420a
- Colab notebook: https://colab.research.google.com/drive/18Ya2CYoC0AxaX3NmjQ_VXO2yp-YCT8oo?usp=sharing

Abstract

We introduce M-CLTR, a measurable latent reasoning framework that separates knowledge quality from control over iterative computation. The system operates over a small action space—continue, overwrite, and restart—and logs internal signals to evaluate reasoning behavior.

Across a sequence of synthetic diagnostics, the control space contains real signal, as evidenced by recurrent oracle improvements, but overall performance is dominated by knowledge quality. Recipe visibility has only a small effect, and the learned controller remains brittle, often overusing restart or failing to outperform simple baselines. In its current form, M-CLTR is best understood as a diagnostic framework for studying latent reasoning control rather than as a complete reasoning solution.

1 Introduction

Recent work on reasoning systems increasingly relies on latent computation rather than purely left-to-right text generation. Yet most such systems treat their internal reasoning trajectories as opaque. M-CLTR asks a narrower question: can a latent reasoning trajectory be explicitly controlled, internally measured, and corrected during execution?

The goal of this paper is not to claim a general reasoning breakthrough, but to test whether a small set of control actions can reveal meaningful structure inside iterative latent computation. In particular, we ask whether:

- control decisions are meaningful rather than arbitrary,
- internal evaluator signals correlate with downstream correctness, and
- reasoning failures can sometimes be detected and repaired by simple interventions.

M-CLTR therefore acts as a plug-in control layer over a base knowledge representation. Its action space is deliberately small:

$$a_t \in \{\text{continue}, \text{overwrite}, \text{restart}\}.$$

This compact design makes the reasoning process measurable enough to study, even if it is not yet strong enough to solve difficult tasks reliably.

2 Method

2.1 Framework

Given an input x, a base model produces an initial knowledge representation,

$$k_0 = K(x), \qquad z_0 = E(k_0),$$

where E maps knowledge into an initial latent reasoning state.

At each iteration, the reasoning layer proposes a candidate update,

$$\tilde{z}_{t+1} = F(z_t, k_0),$$

and an evaluator assigns a scalar score,

$$e_t = V(\tilde{z}_{t+1}).$$

A controller then selects one of three actions,

$$a_t \in \{\text{continue}, \text{overwrite}, \text{restart}\}.$$

The action semantics are:

- **Continue:** $z_{t+1} = \tilde{z}_{t+1}$,
- **Overwrite:** $z_{t+1} = P(\tilde{z}_{t+1})$,
- **Restart:** $z_{t+1} = z_0$.

Conceptually, continue trusts the current latent trajectory, overwrite applies a corrective projection, and restart abandons the current path and returns to the initial state.

2.2 Logged Metrics

The framework is designed to expose simple internal measurements rather than token-level traces. The main logged metrics are:

- task accuracy,
- evaluator score,
- score gap between correct and incorrect outcomes,
- action frequencies, and
- runtime in milliseconds per example.

3 Experimental Setup

3.1 Knowledge Diagnostic

The core isolation experiment evaluates four conditions:

- normal k_0, hidden recipe,
- perfect k_0, hidden recipe,
- normal k_0, visible recipe,
- perfect k_0, visible recipe.

These conditions separate the effect of knowledge quality from the effect of recipe visibility.

3.2 Controller Comparison

We compare six controller modes:

- continue,
- restart,
- fixed project overwrite,
- adaptive overwrite,
- heuristic controller,
- oracle controller.

The oracle is not a deployable controller; it serves as an upper bound on the usefulness of the action space.

3.3 Tasks

All experiments use synthetic rule-switching tasks with progressively harder corruption regimes:

- clean,
- mild corruption,
- medium corruption,
- strong corruption.

These tasks are intentionally simple so that failures can be attributed mainly to knowledge quality and control behavior rather than to broad world knowledge.

4 Results

4.1 Knowledge Isolation

Condition	Continue	Restart	Best fixed O/W	Heuristic	Oracle
Normal k_0, hidden recipe	0.352	0.329	0.350	0.329	0.373
Perfect k_0, hidden recipe	1.000	1.000	1.000	1.000	1.000
Normal k_0, visible recipe	0.344	0.331	0.356	0.330	0.366
Perfect k_0, visible recipe	1.000	1.000	1.000	1.000	1.000

Table 1: Four-condition controller-isolation diagnostic, averaged across seeds and levels.

Table 1 gives the first stable result. When the knowledge representation is made effectively perfect, every mode reaches ceiling performance, regardless of whether the recipe is hidden or visible. Those perfect-knowledge conditions are therefore best interpreted as wiring checks: they show that the latent pipeline and control stack can function when the answer signal is ideal, but they do not test controller quality in a meaningful way.

Under normal knowledge, visible recipe information changes the results only slightly. The main bottleneck remains knowledge quality rather than recipe access. The oracle still outperforms the heuristic controller in the non-ceiling settings, which shows that the action space contains useful control signal even though the learned controller does not yet exploit it robustly.

4.2 Controller-Only Stress Test

Level	Continue	Restart	Project	Adaptive	Heuristic	Oracle
Clean	0.393	0.257	0.382	0.292	0.264	0.398
Mild corruption	0.325	0.335	0.310	0.324	0.326	0.367
Medium corruption	0.398	0.371	0.407	0.367	0.377	0.443
Strong corruption	0.315	0.312	0.312	0.307	0.326	0.359

Table 2: Controller-only stress test on the hardest non-trivial setting: normal k_0 with hidden recipe, averaged across seeds.

Table 2 isolates the controller under the single most relevant regime for this paper: normal k_0 with hidden recipe. The oracle again provides the clearest upper bound, while the heuristic controller

fails to deliver a reliable advantage over continue. Restart is often at least as competitive as overwrite, and overwrite helps only modestly rather than emerging as the dominant recovery operator. This suggests that the control problem is real and measurable, but that the current evaluator-based controller is still too brittle to serve as the main empirical success case.

4.3 Summary by Knowledge Strength

α	Continue	Heuristic	Oracle
0.00	0.344	0.333	0.374
0.33	0.498	0.496	0.566
0.66	0.975	0.973	0.986

Table 3: Final hidden-recipe diagnostic sweep, averaged across three seeds and all difficulty levels.

Table 3 is the clearest final result. Accuracy improves monotonically with knowledge strength. At $\alpha = 0.00$, the system remains strongly knowledge-limited. At $\alpha = 0.33$, the control problem becomes most informative because all methods improve while the oracle gains the most. By $\alpha = 0.66$, the task is already close to saturation, so the experiment begins to lose value as a control benchmark.

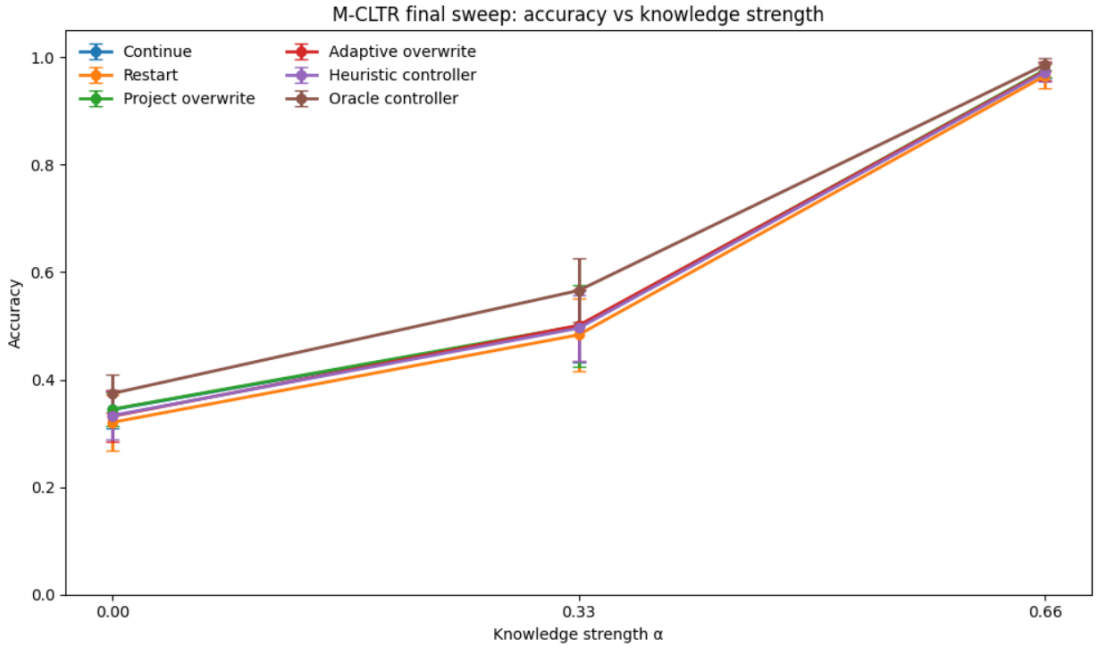

Figure 1: Accuracy versus knowledge strength for the final hidden-recipe sweep. The strongest pattern is monotonic scaling with knowledge quality, while the oracle remains the best controller in the informative non-ceiling regimes.

4.4 Key Observations

1. Knowledge dominates performance.

The strongest signal in the paper is the monotonic improvement with knowledge strength. Once knowledge becomes strong, nearly all controller variants perform well.

2. Oracle improvements show the action space is real.

The oracle controller consistently outperforms heuristic control in the weaker and intermediate knowledge regimes. This demonstrates that continue, overwrite, and restart are meaningful actions rather than arbitrary interventions.

3. The learned controller remains weak.

The heuristic controller often overuses restart, is unstable across seeds, and rarely matches oracle performance. In many settings it is only marginally better than continue, or not better at all.

4. Recipe visibility has minimal impact.

Differences between hidden and visible recipe settings are small relative to the effect of knowledge strength.

5. Overwrite is secondary.

Overwrite provides modest gains in some regimes, but it is not yet more reliable than restart. Restart often acts as the safer recovery move.

4.5 Action Behavior and Speed

At low knowledge strength, restart is common. At intermediate knowledge strength, action behavior becomes mixed and the oracle is most informative. At high knowledge strength, the heuristic controller becomes almost pure continue because the task is already nearly solved.

The timing results are also consistent: continue is fastest, heuristic control adds moderate overhead, and oracle control is slowest because it evaluates multiple futures. Any future learned controller will therefore need to justify additional compute with substantially larger gains than those observed here.

5 Discussion

The results support a narrow but stable conclusion:

> M-CLTR provides a measurable latent-control layer whose action space carries useful signal, but on these toy tasks performance is dominated by knowledge quality and the current learned controller remains brittle.

This reframes the contribution of the work. M-CLTR should not be read as a solved reasoning system, and the present experiments do not show that overwrite is the main repair mechanism or that the learned policy is already strong. Instead, the framework is useful because it makes latent control measurable enough to diagnose where the bottlenecks actually are.

6 Conclusion

M-CLTR demonstrates that:

- latent reasoning can be controlled and measured,
- control actions have real downstream impact,
- knowledge quality dominates outcomes on the current tasks, and
- learned control remains an open problem.

The most defensible claim is therefore diagnostic: M-CLTR is a useful framework for studying reasoning control, but not yet a complete reasoning solution. Future progress depends more on stronger base knowledge representations and improved controller learning than on expanding the current action set.

A Compressed Experimental History

The experimental program began with small latent-control prototypes designed only to answer a basic feasibility question: can a latent trajectory be stepped, scored, and interrupted with explicit actions? These early runs established that the M-CLTR pipeline was instrumentable. Continue, overwrite, and restart could all be logged, evaluator scores could be attached to intermediate states, and corrupted rollouts could be compared under matched settings. At the same time, these first experiments were unstable and highly seed-sensitive, so they were useful mainly for mechanism debugging rather than for substantive claims.

The next phase compared the action types more directly under toy corruption settings. These controller-comparison runs repeatedly showed that restart was often as strong as or stronger than overwrite when the latent state became unreliable. This was an important negative result: it suggested that overwrite should not yet be treated as the signature success of the framework. Instead, the main value of the action space was that distinct interventions could be measured and contrasted at all, even when one of them—restart—turned out to be the safer fallback.

After that, several overwrite variants were explored, including fixed projection rules, adaptive projection, and evaluator-gated heuristics. These experiments asked whether overwrite could become selective enough to repair trajectories without simply reverting to restart behavior. The outcome was mixed. Some overwrite variants produced modest gains in specific regimes, but none emerged as a consistently dominant repair operator. In particular, evaluator-gated and heuristic forms of overwrite were often brittle, which indicated that the main limitation was not the existence of an overwrite action but the weakness of the learned decision rule controlling when to use it.

A separate set of policy-learning experiments focused on whether a simple learned or heuristic controller could outperform passive continue behavior once evaluator signals were available. These runs clarified that the action space contains real signal, but also that the learned controller does not yet extract that signal robustly. The heuristic controller frequently overused restart, behaved inconsistently across seeds, and only occasionally beat continue by a meaningful margin. This phase was therefore important less as a success story than as evidence that controller learning remains the main open problem inside the current framework.

The project then shifted toward knowledge-isolation diagnostics. One important experiment compared normal versus perfect knowledge together with hidden versus visible recipe information. This clarified that knowledge quality is the dominant bottleneck: perfect knowledge made the task trivial regardless of controller mode, while visible recipe information produced only small changes when knowledge remained weak. Those perfect-knowledge conditions were best read as wiring checks rather than controller evaluations, but they still showed that the full latent-control stack can function when the answer signal is effectively ideal.

A related controller-only stress test narrowed the setting further to the hardest non-trivial regime: normal k_0 with hidden recipe. This experiment asked whether a simple evaluator-based controller could improve over continue once the easy knowledge confounds were removed. The answer was only partially positive. Oracle control remained clearly useful, which confirmed that the control space is real, but the learned heuristic controller did not deliver a stable improvement and often collapsed toward restart-heavy behavior. That result made the paper's diagnostic framing much sharper: the

controller problem is measurable, but the current controller is not yet a convincing success case.

The final stage was a multi-seed knowledge-strength sweep under hidden-recipe conditions. Here the knowledge representation was gradually strengthened through $\alpha \in \{0.00, 0.33, 0.66\}$ so that the experiments could ask whether control quality improves smoothly as knowledge improves. This yielded the cleanest monotonic pattern in the whole project. At low knowledge, all methods remained limited. At intermediate knowledge, the oracle gained the most and exposed the available control signal most clearly. At high knowledge, performance approached ceiling for every method, so the task ceased to be informative about controller quality. This final sweep therefore became the main paper result: it demonstrates that M-CLTR is measurable, that the action space matters, and that overall performance is still driven much more by knowledge strength than by the current learned controller.

B Final Interpretation

The full experimental program converges to three stable findings:

- the control space is real, as shown by recurrent oracle advantages,
- knowledge quality dominates performance, and
- the learned controller remains weak.

Taken together, these findings support a restrained interpretation of the paper's contribution. M-CLTR is most valuable not as a finished reasoning architecture, but as an experimental scaffold for separating three questions that are often conflated: whether the system knows the relevant information, whether the latent trajectory can in principle be steered, and whether a practical controller can learn to apply that steering reliably. On the present synthetic tasks, the first question dominates the final accuracy numbers, the second receives a clear positive answer from the oracle comparisons, and the third remains unresolved.

That combination of results is still meaningful. A framework that makes internal control measurable, exposes when restart is safer than overwrite, and reveals how quickly strong knowledge can wash out controller differences is useful for future iteration. The cleanest conclusion is therefore methodological: M-CLTR succeeds as a diagnostic lens on latent reasoning control, while the construction of a robust learned controller remains the central unfinished step.